

Recenzja kodu Projektu na przedmiot Programowanie Obiektowe

Projekt Wypożyczalnia pojazdów wykonany przez Patryka Czupaka i Konrada Lipowskiego

Projekt to system wypożyczalni pojazdów napisany w C++. Aplikacja działa w konsoli i pozwala m.in. dodawać pojazdy oraz klientów, listować ich, a także wykonywać wypożyczenie i zwrot. Za komunikację z użytkownikiem odpowiada main.cpp (proste komendy w CLI). Główna logika jest w klasie RentalService, która zarządza:

- listą klientów i pojazdów,
- transakcjami wypożyczeń (RentalTransaction),
- sprawdzeniem dostępności w czasie (zakres dat).

1. Wzorce projektowe wykorzystane w projekcie

• Singleton

Wzorzec Singleton został użyty w RentalService (metoda getInstance()), żeby w programie istniała jedna główna instancja klasy zarządzającej działaniem wypożyczalni.

• Factory

Wzorzec Factory jest widoczny w VehicleFactory, która tworzy różne typy pojazdów (Car, Truck, Motorcycle). Dzięki temu tworzenie obiektów jest w jednym miejscu, a reszta kodu nie musi znać szczegółów konstruktorów.

• Observer

Wzorzec Observer jest użyty do zdarzeń wypożyczeń. RentalService powiadamia obserwatorów (IRentalObserver) o ważnych akcjach, np. wypożyczeniu lub zwrocie. Przykłady: ConsoleLoggerObserver i StatsObserver.

2. Elementy programowania obiektowego

- **Dziedziczenie i polimorfizm:** Vehicle jest klasą bazową, a Car/Truck/Motorcycle ją rozszerzają i mają własne implementacje np. liczenia ceny (calculatePrice()).
- **Enkapsulacja:** dane klas są ukryte, a dostęp odbywa się przez metody (np. informacje o pojeździe, status, cena).
- **Agregacja/kompozycja:** RentalService przechowuje kolekcję obiektów (pojazdy, klienci, transakcje) i nimi zarządza.

3. Testy i uruchomienie

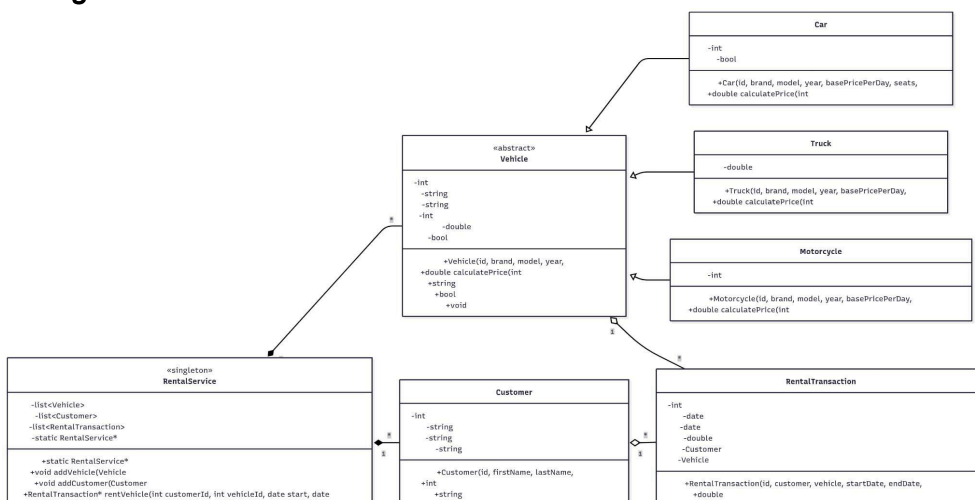
Projekt zawiera podstawowe testy w tests/test_basic.cpp. Budowanie i testowanie jest zrobione przez CMake, a dodatkowo są skrypty typu ci.sh, które ułatwiają ich uruchamianie.

4. Krótka ocena

Projekt jest spójny i spełnia wymagania PO: używa OOP, ma dziedziczenie i polimorfizm, zawiera wzorce projektowe (Singleton/Factory/Observer) oraz ma testy.

Do ewentualnej poprawy można dodać więcej walidacji danych (np. duplikaty ID, poprawność dat), ale na etap projektu studenckiego całość jest OK.

5. Diagram UML



6. Link do githuba

https://github.com/dataproctech/project-po-CzupakCzupak/tree/main/rental_m2_clean